DESIGN & ANALYSIS OF ALGORITHMS

Algorithms in C
A Practical Reference

Fourteen classic algorithms implemented in C — sorting, graph traversal, shortest paths, minimum spanning trees, string matching, dynamic programming, backtracking and branch & bound — with clean, consistently formatted, syntax highlighted source code for study and lab reference.

CONTENTS

Table of Contents

01

SORTING · DIVIDE & CONQUER

Merge Sort

TIME COMPLEXITY	SPACE COMPLEXITY	STABLE
O(n log n) — best, average and worst case	**O(n) auxiliary array for merging**	**Yes**

Key idea: Split the array into two halves, sort each half recursively, then **merge** the two sorted halves back together by repeatedly picking the smaller of the two front elements. Because the array is always split exactly in half, the recursion depth is $\log_2(n)$ and merging at every level costs O(n), giving the guaranteed O(n log n) bound regardless of the input order.

01_merge_sort.c — 97 lines · C

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 500000
void merge(int a[], int low, int mid, int high)
{
    int i = low, j = mid + 1, k = 0;
    int c[MAX];
    while (i <= mid && j <= high)
    {
        if (a[i] <= a[j])
            c[k++] = a[i++];
        else
            c[k++] = a[j++];
    }
    while (i <= mid)
        c[k++] = a[i++];
    while (j <= high)
        c[k++] = a[j++];
    for (i = low, j = 0; i <= high; i++, j++)
        a[i] = c[j];
}
void mergeSort(int a[], int low, int high)
{
    if (low >= high)
        return;
    int mid = (low + high) / 2;
    mergeSort(a, low, mid);
    mergeSort(a, mid + 1, high);
    merge(a, low, mid, high);
}
int main()
{
    int a[MAX];
    int i, n, ch, value, p;
    FILE *fp;
    clock_t start, end;
    double exetime;
    while (1)
    {
        printf("\n===== MERGE SORT MENU =====\n");
```

```c
        printf("1. Merge Sort\n");
        printf("2. Record Execution Time\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &ch);
        switch (ch)
        {
            case 1:
                printf("Enter number of elements: ");
                scanf("%d", &n);
                fp = fopen("input.txt", "w");
                for (i = 0; i < n; i++)
                {
                    value = rand() % 10000;
                    fprintf(fp, "%d\n", value);
                }
                fclose(fp);
                printf("Random numbers stored in input.txt\n");
                fp = fopen("input.txt", "r");
                i = 0;
                for (i = 0; i < n; i++)
                {
                    fscanf(fp, "%d", &a[i]);
                }
                fclose(fp);
                mergeSort(a, 0, n - 1);
                fp = fopen("output.txt", "w");
                for (i = 0; i < n; i++)
                    fprintf(fp, "%d\n", a[i]);
                fclose(fp);
                printf("Sorted elements stored in output.txt\n");
                break;
            case 2:
                fp = fopen("plot.dat", "w");
                for (p = 10000; p <= 100000; p += 10000)
                {
                    for (i = 0; i < p; i++)
                        a[i] = rand() % 10000;
                    start = clock();
                    mergeSort(a, 0, p - 1);
                    end = clock();
                    exetime = (double)(end - start) / CLOCKS_PER_SEC;
                    printf("N = %d\tTime = %lf seconds\n", p, exetime);
                    fprintf(fp, "%d %lf\n", p, exetime);
                }
                fclose(fp);
                printf("Execution times stored in plot.dat\n");
                break;
            case 3:
                exit(0);
            default:
                printf("Invalid choice!\n");
        }
    }
    return 0;
}
```

02

SORTING · DIVIDE & CONQUER

Quick Sort

TIME COMPLEXITY	SPACE COMPLEXITY	STABLE
O(n log n) average, O(n²) worst case	**O(log n) recursion stack (in-place)**	**No**

Key idea: Pick a **pivot** (here, the first element) and partition the array so everything smaller ends up on its left and everything larger on its right, then recursively sort each side. The partition step does the real work in-place using two scanning pointers, which is what makes Quick Sort fast in practice despite its O(n²) worst case on already-sorted or adversarial input.

02_quick_sort.c — 112 lines · C

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 500000
void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}
int partition(int a[], int low, int high)
{
    int pivot = a[low];
    int i = low;
    int j = high;
    while (i < j)
    {
        while (i <= high - 1 && pivot >= a[i])
        {
            i++;
        }
        while (j >= low + 1 && pivot < a[j])
        {
            j--;
        }
        if (i < j)
        {
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[low], &a[j]);
    return j;
}
void QuickSort(int a[], int low, int high)
{
    if (low >= high)
    {
        return;
    }
    int s = partition(a, low, high);
    QuickSort(a, low, s - 1);
```

```c
    QuickSort(a, s + 1, high);
}
int main()
{
    int a[MAX];
    int i, n, ch, value, p;
    FILE *fp;
    clock_t start, end;
    double exetime;
    while (1)
    {
        printf("\n===== QUICK SORT MENU =====\n");
        printf("1. Quick Sort\n");
        printf("2. Record Execution Time\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &ch);
        switch (ch)
        {
            case 1:
                printf("Enter number of elements: ");
                scanf("%d", &n);
                fp = fopen("input.txt", "w");
                for (i = 0; i < n; i++)
                {
                    value = rand() % 10000;
                    fprintf(fp, "%d\n", value);
                }
                fclose(fp);
                printf("Random numbers stored in input.txt\n");
                fp = fopen("input.txt", "r");
                for (i = 0; i < n; i++)
                {
                    fscanf(fp, "%d", &a[i]);
                }
                fclose(fp);
                QuickSort(a, 0, n - 1);
                fp = fopen("output.txt", "w");
                for (i = 0; i < n; i++)
                {
                    fprintf(fp, "%d\n", a[i]);
                }
                fclose(fp);
                printf("Sorted elements stored in output.txt\n");
                break;
            case 2:
                fp = fopen("plot.dat", "w");
                for (p = 10000; p <= 100000; p += 10000)
                {
                    for (i = 0; i < p; i++)
                    {
                        a[i] = rand() % 10000;
                    }
                    start = clock();
                    QuickSort(a, 0, p - 1);
                    end = clock();
                    exetime = (double)(end - start) / CLOCKS_PER_SEC;
                    printf("N = %d\tTime = %lf seconds\n", p, exetime);
                    fprintf(fp, "%d %lf\n", p, exetime);
                }
                fclose(fp);
```

```
                printf("Execution times stored in plot.dat\n");
                break;
            case 3:
                exit(0);
            default:
                printf("Invalid Choice!\n");
        }
    }
    return 0;
}
```

03

GRAPH TRAVERSAL · QUEUE-BASED

Breadth First Search

TIME COMPLEXITY	SPACE COMPLEXITY	EXPLORES
O(V²) for an adjacency matrix	**O(V) for the queue and visited array**	**Level by level**

Key idea: Starting from a source node, visit all of its direct neighbours first, then all of *their* unvisited neighbours, and so on — expanding outward one layer at a time. A simple array-based **queue** (front/rear indices) keeps track of which node to explore next, guaranteeing nodes are reported in increasing distance from the start.

03_bfs.c — 51 lines · C

```c
#include <stdio.h>
#define MAX 100 // Maximum number of vertices
void bfs(int graph[MAX][MAX], int visited[MAX], int n, int start)
{
    int q[MAX];        // Simulated queue using an array
    int f = 0, r = -1; // Indices for front and rear of the queue
    visited[start] = 1;
    q[++r] = start; // Enqueue the starting node
    while (f <= r)
    {
        int cur = q[f++];
        printf("%d ", cur);
        for (int i = 0; i < n; ++i)
        {
            if (graph[cur][i] && !visited[i])
            {
                visited[i] = 1;
                q[++r] = i; // Enqueue adjacent nodes
            }
        }
    }
    // Print nodes that are not reachable
    printf("\nNodes not reachable from node %d: ", start);
    for (int i = 0; i < n; ++i)
    {
        if (!visited[i])
        {
            printf("%d ", i);
        }
    }
}
int main()
{
    int n, s, graph[MAX][MAX], visited[MAX] = {0};
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
```

```
            scanf("%d", &graph[i][j]);
        }
    }
    printf("Enter starting node: ");
    scanf("%d", &s);
    printf("Nodes reachable from node %d in BFS order:\n", s);
    bfs(graph, visited, n, s);
    printf("\n");
    return 0;
}
```

04

GRAPH TRAVERSAL · RECURSIVE

Depth First Search

TIME COMPLEXITY	SPACE COMPLEXITY	EXPLORES
O(V²) for an adjacency matrix	**O(V) recursion stack and visited array**	**Branch by branch**

Key idea: Starting from a source node, plunge as deep as possible along one path before backtracking — visit a node, then **recursively** visit the first unvisited neighbour, and only return to try the next neighbour once that entire branch is exhausted. The call stack implicitly remembers the path back.

04_dfs.c — 34 lines · C

```c
#include <stdio.h>
#define MAX 100 // Maximum number of vertices
void dfs(int graph[MAX][MAX], int visited[MAX], int n, int node)
{
    printf("%d ", node);
    visited[node] = 1;
    for (int i = 0; i < n; i++)
    {
        if (graph[node][i] && !visited[i])
            dfs(graph, visited, n, i);
    }
}
int main()
{
    int n, start, graph[MAX][MAX], visited[MAX] = {0};
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);
    printf("Enter starting node: ");
    scanf("%d", &start);
    printf("Nodes reachable from node %d in DFS order:\n", start);
    dfs(graph, visited, n, start);
    printf("\n nodes that are not reachable from %d:\n", start);
    for (int i = 0; i < n; i++)
    {
        if (!visited[i])
            printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

05

GRAPH ORDERING · KAHN'S ALGORITHM

Topological Sort

TIME COMPLEXITY	SPACE COMPLEXITY	REQUIRES
O(V²) for an adjacency matrix	**O(V) for in-degree array and stack**	**A directed acyclic graph (DAG)**

Key idea: Repeatedly pick a vertex with **in-degree zero** (no remaining prerequisites), output it, and remove its outgoing edges — which may free up new zero-in-degree vertices. This is Kahn's algorithm: the order produced respects every directed edge u→v by always placing u before v.

05_topological_sort.c — 58 lines · C

```c
#include <stdio.h>
void topological(int a[10][10], int n)
{
    int in[10], out[10], stack[10], top = -1;
    int i, j, k = 0;
    for (i = 0; i < n; i++)
    {
        in[i] = 0;
        for (j = 0; j < n; j++)
        {
            if (a[j][i] == 1)
                in[i]++;
        }
    }
    while (1)
    {
        for (i = 0; i < n; i++)
        {
            if (in[i] == 0)
            {
                stack[++top] = i;
                in[i] = -1;
            }
        }
        if (top == -1)
            break;
        out[k] = stack[top--];

        for (i = 0; i < n; i++)
        {
            if (a[out[k]][i] == 1)
                in[i]--;
        }
        k++;
    }
    printf("\nTopological Sorting is:\n");
    for (i = 0; i < k; i++)
    {
        printf("%d ", out[i] + 1);
    }
}
int main()
```

```c
{
    int a[10][10], n, i, j;
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    printf("Enter the adjacency matrix:\n");
    for (i = 0; i < n; i++)
    {
        printf("Enter row %d:\n", i + 1);
        for (j = 0; j < n; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
    topological(a, n);
    return 0;
}
```

06

SORTING · BINARY HEAP

Heap Sort

TIME COMPLEXITY	SPACE COMPLEXITY	STABLE
O(n log n) — best, average and worst case	**O(1) extra — sorts in-place**	**No**

Key idea: First arrange the array into a **max-heap** so the largest element sits at index 0. Then repeatedly swap that root with the last unsorted element and **heapify** the shrinking heap to restore the max-heap property — each swap permanently places one more element in its final sorted position.

06_heap_sort.c — 113 lines · C

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 500000
void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}
void heapify(int a[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && a[left] > a[largest])
    {
        largest = left;
    }
    if (right < n && a[right] > a[largest])
    {
        largest = right;
    }
    if (largest != i)
    {
        swap(&a[i], &a[largest]);
        heapify(a, n, largest);
    }
}
void heapSort(int a[], int n)
{
    int i;
    // Build Max Heap
    for (i = n / 2 - 1; i >= 0; i--)
    {
        heapify(a, n, i);
    }
    // Heap Sort
    for (i = n - 1; i >= 1; i--)
    {
        swap(&a[0], &a[i]);
        heapify(a, i, 0);
```

```c
    }
}
int main()
{
    int a[MAX];
    int i, n, ch, value, p;
    FILE *fp;
    clock_t start, end;
    double exetime;
    while (1)
    {
        printf("\n===== HEAP SORT MENU =====\n");
        printf("1. Heap Sort\n");
        printf("2. Record Execution Time\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &ch);
        switch (ch)
        {
            case 1:
                printf("Enter number of elements: ");
                scanf("%d", &n);
                fp = fopen("input.txt", "w");
                for (i = 0; i < n; i++)
                {
                    value = rand() % 10000;
                    fprintf(fp, "%d\n", value);
                }
                fclose(fp);
                printf("Random numbers stored in input.txt\n");
                fp = fopen("input.txt", "r");
                for (i = 0; i < n; i++)
                {
                    fscanf(fp, "%d", &a[i]);
                }
                fclose(fp);
                heapSort(a, n);
                fp = fopen("output.txt", "w");
                for (i = 0; i < n; i++)
                {
                    fprintf(fp, "%d\n", a[i]);
                }
                fclose(fp);
                printf("Sorted elements stored in output.txt\n");
                break;
            case 2:
                fp = fopen("plot.dat", "w");
                for (p = 10000; p <= 100000; p += 10000)
                {
                    for (i = 0; i < p; i++)
                    {
                        a[i] = rand() % 10000;
                    }
                    start = clock();
                    heapSort(a, p);
                    end = clock();
                    exetime = (double)(end - start) / CLOCKS_PER_SEC;
                    printf("N = %d\tTime = %lf seconds\n", p, exetime);
                    fprintf(fp, "%d %lf\n", p, exetime);
                }
                fclose(fp);
```

```
                printf("Execution times stored in plot.dat\n");
                break;
            case 3:
                exit(0);
            default:
                printf("Invalid Choice!\n");
        }
    }
    return 0;
}
```

07

SHORTEST PATH · DYNAMIC PROGRAMMING

Floyd–Warshall Algorithm

TIME COMPLEXITY	SPACE COMPLEXITY	FINDS
$O(V^3)$	$O(V^2)$ for the distance matrix	All-pairs shortest distances

Key idea: For every intermediate vertex k, ask whether routing a path through k makes it shorter: `D[i][j] = min(D[i][j], D[i][k] + D[k][j])`. Doing this for every pair (i, j) and every possible intermediate k gradually relaxes every distance down to its true shortest-path value across the whole graph.

07_floyds.c — 39 lines · C

```c
#include <stdio.h>
#include <stdlib.h>
int MIN(int v1, int v2)
{
    if (v1 >= v2)
        return v2;
    else
        return v1;
}
int main()
{
    int W[10][10], D[10][10], n, i, j, k;
    printf("Enter the numbner of Vertices :");
    scanf("%d", &n);
    printf("Enter the Weight matrix of the given graph row wise \n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
        {
            scanf("%d", &W[i][j]);
            D[i][j] = W[i][j];
        }
    for (k = 0; k < n; k++)
    {
        for (i = 0; i < n; i++)
        {
            for (j = 0; j < n; j++)
                D[i][j] = MIN(D[i][j], D[i][k] + D[k][j]);
        }
    }
    printf("Solution to All pairs shortest distance problem\n");
    printf("Using Floyd's Algorithm - Distance Matrix of the given graph is \n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
            printf("%d\t", D[i][j]);
        printf("\n");
    }
    return 0;
}
```

08

SHORTEST PATH · GREEDY

Dijkstra's Algorithm

TIME COMPLEXITY	SPACE COMPLEXITY	REQUIRES
O(V²) for an adjacency matrix	**O(V) for distance and visited arrays**	**Non-negative edge weights**

Key idea: Maintain a tentative distance to every vertex, starting at 0 for the source and infinity elsewhere. Repeatedly pick the closest **unvisited** vertex, lock in its distance, and **relax** its neighbours' distances through it. Because weights are non-negative, once a vertex is visited its shortest distance can never improve.

08_dijkstra.c — 71 lines · C

```c
#include <stdio.h>
#define INFINITY 9999
#define MAX 10
void Dijkstra(int Graph[MAX][MAX], int n, int start)
{
    int cost[MAX][MAX], distance[MAX], visited[MAX];
    int count, mindistance, nextnode, i, j;
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (Graph[i][j] == 0)
                cost[i][j] = INFINITY;
            else
                cost[i][j] = Graph[i][j];
        }
    }
    for (i = 0; i < n; i++)
    {
        distance[i] = cost[start][i];
        visited[i] = 0;
    }
    distance[start] = 0;
    visited[start] = 1;
    count = 1;
    while (count < n - 1)
    {
        mindistance = INFINITY;
        for (i = 0; i < n; i++)
        {
            if (distance[i] < mindistance && !visited[i])
            {
                mindistance = distance[i];
                nextnode = i;
            }
        }
        visited[nextnode] = 1;
        for (i = 0; i < n; i++)
        {
            if (!visited[i] &&
                mindistance + cost[nextnode][i] < distance[i])
            {
```

```
                distance[i] = mindistance + cost[nextnode][i];
            }
        }
        count++;
    }
    printf("\nShortest Distances:\n");
    for (i = 0; i < n; i++)
    {
        if (i != start)
            printf("0 -> %d = %d\n", i, distance[i]);
    }
}
int main()
{
    int Graph[MAX][MAX];
    int n, i, j;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &Graph[i][j]);
        }
    }
    Dijkstra(Graph, n, 0);
    return 0;
}
```

09

MINIMUM SPANNING TREE · GREEDY

Kruskal's Algorithm

TIME COMPLEXITY	SPACE COMPLEXITY	BUILDS THE MST
O(V²) here (scans the cost matrix each step)	**O(V) for the union-find parent array**	**Edge by edge, cheapest first**

Key idea: Sort — or in this matrix version, repeatedly scan for — the cheapest remaining edge, and add it to the spanning tree only if its two endpoints aren't already connected. A simple **union–find** (`find` / `uni`) tracks connected components so cycle-forming edges are skipped.

09_kruskals.c — 62 lines · C

```c
#include <stdio.h>
int i, j, a, b, u, v, n, ne = 1, min, Mcost = 0;
int cost[10][10], parent[10];
int find(int i)
{
    while (parent[i])
        i = parent[i];
    return i;
}
int uni(int i, int j)
{
    if (i != j)
    {
        parent[j] = i;
        return 1;
    }
    return 0;
}
int main()
{
    printf("\n\nImplementation of Kruskal's Algorithm\n\n");
    printf("Input the number of vertices: ");
    scanf("%d", &n);
    printf("Input the adjacency matrix for the graph:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0)
                cost[i][j] = 999;
        }
    }
    printf("\nThe edges of MST are:\n");
    printf("\nEdge\tWeight\n");
    while (ne < n)
    {
        min = 999;
        for (i = 0; i < n; i++)
        {
            for (j = 0; j < n; j++)
            {
```

```
                if (cost[i][j] < min)
                {
                    min = cost[i][j];
                    a = u = i;
                    b = v = j;
                }
            }
        }
        u = find(u);
        v = find(v);
        if (uni(u, v))
        {
            printf("%d edge (%d - %d) = %d\n", ne++, a, b, min);
            Mcost += min;
        }
        cost[a][b] = cost[b][a] = 999;
    }
    printf("\nCost of the Minimum Spanning Tree (MST) = %d\n", Mcost);
    return 0;
}
```

10

MINIMUM SPANNING TREE · GREEDY

Prim's Algorithm

TIME COMPLEXITY	SPACE COMPLEXITY	BUILDS THE MST
O(V²) for an adjacency matrix	**O(V) for key, parent and mstSet arrays**	**Growing one tree outward**

Key idea: Start the spanning tree from any single vertex, then repeatedly attach the **cheapest edge** that connects a vertex already in the tree to one that isn't. Unlike Kruskal's, the tree always stays a single connected piece, growing outward one edge at a time.

10_prims.c — 71 lines · C

```c
#include <stdio.h>
#define MAX 20
int minKey(int key[], int mstSet[], int n)
{
    int min = 999, minIndex;
    for (int i = 0; i < n; i++)
    {
        if (!mstSet[i] && key[i] < min)
        {
            min = key[i];
            minIndex = i;
        }
    }
    return minIndex;
}
void printMST(int parent[], int graph[MAX][MAX], int n)
{
    int cost = 0;
    printf("\nEdge\tWeight\n");
    for (int i = 1; i < n; i++)
    {
        printf("%d - %d\t%d\n", parent[i], i, graph[i][parent[i]]);
        cost += graph[i][parent[i]];
    }
    printf("Cost of MST = %d\n", cost);
}
void primMST(int graph[MAX][MAX], int n)
{
    int parent[MAX];
    int key[MAX];
    int mstSet[MAX];
    for (int i = 0; i < n; i++)
    {
        key[i] = 999;
        mstSet[i] = 0;
    }
    key[0] = 0;
    parent[0] = -1;
    for (int count = 0; count < n - 1; count++)
    {
        int u = minKey(key, mstSet, n);
        mstSet[u] = 1;
```

```
        for (int v = 0; v < n; v++)
        {
            if (graph[u][v] && !mstSet[v] &&
                graph[u][v] < key[v])
            {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }
    printMST(parent, graph, n);
}
int main()
{
    int n;
    int graph[MAX][MAX];
    printf("Input the number of vertices: ");
    scanf("%d", &n);
    printf("Input the adjacency matrix:\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }
    primMST(graph, n);
    return 0;
}
```

11

DYNAMIC PROGRAMMING · TABULATION

0/1 Knapsack

TIME COMPLEXITY	SPACE COMPLEXITY	VARIANT
O(n · W)	**O(n · W) for the DP table**	**0/1 — each item used at most once**

Key idea: Build a table `V[i][j]` holding the best profit using the first i items with capacity j. For each item, either **leave it out** (reuse the row above) or **take it** (add its profit to the best solution for the remaining capacity) — whichever value is larger. Walking the filled table backwards recovers which items were chosen.

11_knapsack.c — 59 lines · C

```c
#include <stdio.h>
int n, W;
int w[10], v[10], x[10], V[10][10];
int MAX(int a, int b)
{
    return (a > b) ? a : b;
}
void Knapsack()
{
    int i, j;
    for (i = 0; i <= n; i++)
    {
        for (j = 0; j <= W; j++)
        {
            if (i == 0 || j == 0)
                V[i][j] = 0;
            else if (j < w[i])
                V[i][j] = V[i - 1][j];
            else
                V[i][j] = MAX(V[i - 1][j], V[i - 1][j - w[i]] + v[i]);
            printf("%3d", V[i][j]);
        }
        printf("\n");
    }
}
void PrintSolution()
{
    int i = n, j = W;
    while (i > 0 && j > 0)
    {
        if (V[i][j] != V[i - 1][j])
        {
            x[i] = 1;
            j -= w[i];
        }
        i--;
    }
}
int main()
{
    int i;
    printf("Enter number of objects: ");
```

```
    scanf("%d", &n);
    printf("Enter knapsack capacity: ");
    scanf("%d", &W);
    printf("Enter weight and profit:\n");
    for (i = 1; i <= n; i++)
        scanf("%d%d", &w[i], &v[i]);
    printf("\nSolution Matrix:\n");
    Knapsack();
    PrintSolution();
    printf("\nSelected Objects:\n");
    printf("Object\tWeight\tProfit\n");
    for (i = 1; i <= n; i++)
        if (x[i])
            printf("%d\t%d\t%d\n", i, w[i], v[i]);
    printf("\nMaximum Profit = %d\n", V[n][W]);
    return 0;
}
```

12

BACKTRACKING · CONSTRAINT SEARCH

N–Queens

TIME COMPLEXITY	SPACE COMPLEXITY	GOAL
O(N!) worst case	**O(N) for the board / column array**	**Place N queens, no two attacking**

Key idea: Place queens one row at a time. Before placing a queen in a column, **check** that no earlier queen shares that column or either diagonal. If a row has no safe column left, **backtrack** to the previous row and try its next option — exploring the search tree only as deep as constraints allow.

12_nqueens.c — 76 lines · C

```c
#include <math.h>
#include <stdio.h>
#include <stdlib.h>

int board[20], count;

int main()
{
    int n, i, j;
    void queen(int row, int n);

    printf(" - N Queens Problem Using Backtracking -");
    printf("\n\nEnter number of Queens:");
    scanf("%d", &n);
    if (n == 2 || n == 3)
        printf("\nSolution doesnot exits \n");
    queen(1, n);
    return 0;
}

// function for printing the solution
void print(int n)
{
    int i, j;
    printf("\n\nSolution %d:\n\n", ++count);

    for (i = 1; i <= n; ++i)
        printf("\t%d", i);

    for (i = 1; i <= n; ++i)
    {
        printf("\n\n%d", i);
        for (j = 1; j <= n; ++j) // for nxn board
        {
            if (board[i] == j)
                printf("\tQ"); // queen at i,j position
            else
                printf("\t-"); // empty slot
        }
    }
    printf("\n");
}
```

```c

/*funtion to check conflicts
If no conflict for desired postion returns 1 otherwise returns 0*/
int place(int row, int column)
{
    int i;
    for (i = 1; i <= row - 1; ++i)
    {
        // checking column and digonal conflicts
        if (board[i] == column)
            return 0;
        else if (abs(board[i] - column) == abs(i - row))
            return 0;
    }

    return 1; // no conflicts
}

// function to check for proper positioning of queen
void queen(int row, int n)
{
    int column;
    for (column = 1; column <= n; ++column)
    {
        if (place(row, column))
        {
            board[row] = column; // no conflicts so place queen
            if (row == n)        // dead end
                print(n);        // printing the board configuration
            else                 // try queen with next position
                queen(row + 1, n);
        }
    }
}
```

13

STRING MATCHING · BAD-CHARACTER SHIFT

Horspool's Algorithm

TIME COMPLEXITY	SPACE COMPLEXITY	SKIP RULE
O(n) average case, O(n · m) worst case	**O(σ) for the shift table (σ = alphabet size, 256 here)**	**Bad-character shift, not a position-by-position scan**

Key idea: Align the pattern against a window of the text and compare **right to left** starting from the last character. On a mismatch, don't just slide one position — look up a precomputed **shift table** for the text character currently under the pattern's last position, and jump the window ahead by that amount, skipping alignments that can't possibly match.

13_horspool.c — 42 lines · C

```c
#include <stdio.h>
#include <string.h>
#define MAX 256
void shiftTable(char p[], int t[])
{
    int m = strlen(p);
    for (int i = 0; i < MAX; i++)
        t[i] = m;
    for (int i = 0; i < m - 1; i++)
        t[p[i]] = m - 1 - i;
}
int horspool(char s[], char p[], int t[])
{
    int n = strlen(s), m = strlen(p);
    int i = m - 1;
    while (i < n)
    {
        int k = 0;
        while (k < m && p[m - 1 - k] == s[i - k])
            k++;
        if (k == m)
            return i - m + 1;
        i += t[s[i]];
    }
    return -1;
}
int main()
{
    char text[MAX], pattern[MAX];
    int table[MAX], pos;
    printf("Enter text: ");
    scanf("%s", text);
    printf("Enter pattern: ");
    scanf("%s", pattern);
    shiftTable(pattern, table);
    pos = horspool(text, pattern, table);
    if (pos == -1)
        printf("Pattern not found");
    else
        printf("Pattern found at position %d", pos + 1);
```

```
    return 0;
}
```

14

BRANCH & BOUND · OPTIMAL ASSIGNMENT

Job Assignment Problem

TIME COMPLEXITY	SPACE COMPLEXITY	GOAL
O(n!) worst case — pruned heavily in practice	**O(n) for the assignment / visited arrays**	**Assign each person one job at minimum total cost**

Key idea: Assign people to jobs one at a time, and before recursing further, compute a **lower bound** on the best total cost reachable from this partial assignment (the cheapest remaining job for every person not yet assigned). If that bound is already worse than the best complete solution found so far, **prune** the branch immediately — this is what avoids exploring all n! assignments.

14_assignment.c — 73 lines · C

```c
#include <stdio.h>
int cost[20][20];
int assigned[20] = {0};
int assign[20];     // Current assignment
int bestAssign[20]; // Best assignment
int n, minCost = 999;
void assignment(int person, int currCost)
{
    int job, i, j;
    int bound = currCost;
    // Calculate Lower Bound
    for (i = person; i < n; i++)
    {
        int min = 999;
        for (j = 0; j < n; j++)
        {
            if (assigned[j] == 0 && cost[i][j] < min)
            {
                min = cost[i][j];
            }
        }
        bound += min;
    }
    // Branch and Bound
    if (bound >= minCost)
    {
        return;
    }
    // All persons assigned
    if (person == n)
    {
        minCost = currCost;
        for (i = 0; i < n; i++)
        {
            bestAssign[i] = assign[i];
        }
        return;
    }
    // Try every unassigned job
    for (job = 0; job < n; job++)
    {
```

```c
        if (assigned[job] == 0)
        {
            assigned[job] = 1;
            assign[person] = job;
            assignment(person + 1,
                       currCost + cost[person][job]);
            assigned[job] = 0;
        }
    }
}
int main()
{
    int i, j;
    printf("Enter number of persons/jobs: ");
    scanf("%d", &n);
    printf("Enter cost matrix:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);
        }
    }
    assignment(0, 0);
    printf("\nMinimum Cost = %d\n", minCost);
    printf("\nOptimal Assignment:\n");
    for (i = 0; i < n; i++)
    {
        printf("Person %d --> Job %d\n", i, bestAssign[i]);
    }
    return 0;
}
```

End of document · Algorithms in C · 14 programs